Объектно-ориентированное программирование

4 семестр
Лекция 3.

Содержание лекции:

- Тестирование
- Виды ошибок
- Уровни тестирования
- Модульные тесты NUnit

Преподаватель курса: Клюкин Даниил Анатольевич, ст. преподаватель каф. ПМиИТ

Тестирование

Тести́рование програ́ммного обеспе́че́ния — процесс исследования, испытания программного продукта, имеющий своей целью проверку соответствия между реальным поведением программы и её ожидаемым поведением на конечном наборе тестов, выбранных определённым образом.

Верификация (verification) – Соответствие продукта требованиям (**спецификации**). Т.е. Верификация это проверка того, что программа работает так, как было задумано.

Валидация (validation) – Соответствие продукта потребностям пользователей. Т.е. работа программы устраивает заказчика.

Виды ошибок

Error – это ошибка **пользователя**, то есть он пытается использовать программу иным способом (например, вводит буквы в поля, где требуется вводить цифры). В качественной программе предусмотрены такие ситуации и выдаются сообщение об ошибке.

Bug (defect) – это ошибка **программиста** (или дизайнера или ещё кого, кто принимает участие в разработке), то есть когда в программе, что-то идёт не так, как планировалось. Например, внутри программа построена так, что изначально не соответствует тому, что от неё ожидается.

Failure – это сбой в работе компонента, всей программы или системы (может быть как аппаратным, так и вызванным дефектом).

Уровни тестирования

Модульное (Unit) тестирование — тестируется минимальный компонент (класс, метод). Пишут программисты.

Интеграционное тестирование — проверка взаимодействия между несколькими модулями или системами (например, работа с БД).

Системное тестирование — проверка всей системы в сборе на соответствие требованиям (включая UI, нагрузочное тестирование и т.д.).

Приемочное (Альфа/Бета) тестирование — проверка программы на стороне конечных пользователей.

Техники тест-дизайна

Эквивалентное разделение: Разбиваем все возможные данные на группы, где программа ведет себя одинаково. Берем по 1 значению из группы. *(Пример: пароль от 5 до 10 символов. Группы: <5, от 5 до 10, >10. Тестируем длины 3, 7 и 12).*

Анализ граничных значений: Ошибки чаще всего прячутся на границах. *(Для интервала 1-10 проверяем 0, 1, 2 и 9, 10, 11).*

Тестирование компонентов

Модульное (компонентное) тестирование **(Unit Testing)** проводится самими разработчиками, т.к. предполагает полный доступ к коду, для тестирования какого-либо одного логически выделенного и изолированного элемента (модуля) системы в коде, проверяет функциональность и ищет дефекты в частях приложения, которые доступны и могут быть протестированы по-отдельности (модули программ, объекты, классы, функции и т.д.).

Структура модульного теста: Паттерн AAA

Любой хороший тест визуально и логически делится на 3 части:

Arrange (Подготовка): Инициализация объектов, настройка зависимостей, подготовка входных данных.

Act (Действие): Вызов тестируемого метода.

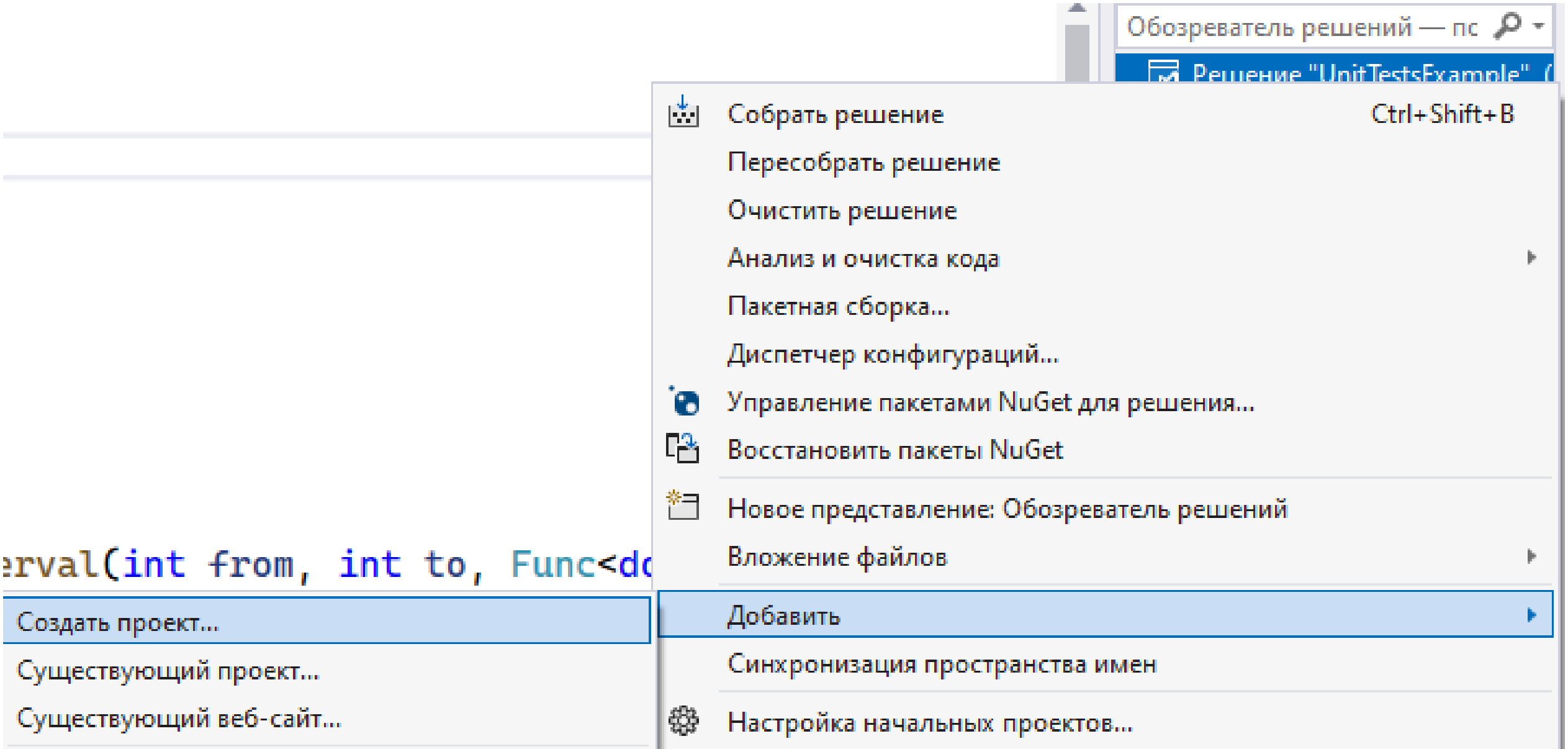
Assert (Утверждение/Проверка): Сравнение полученного результата с ожидаемым.

Создадим WindowsForms проект, добавим класс поиска целочисленных корней уравнения, который будем тестировать:

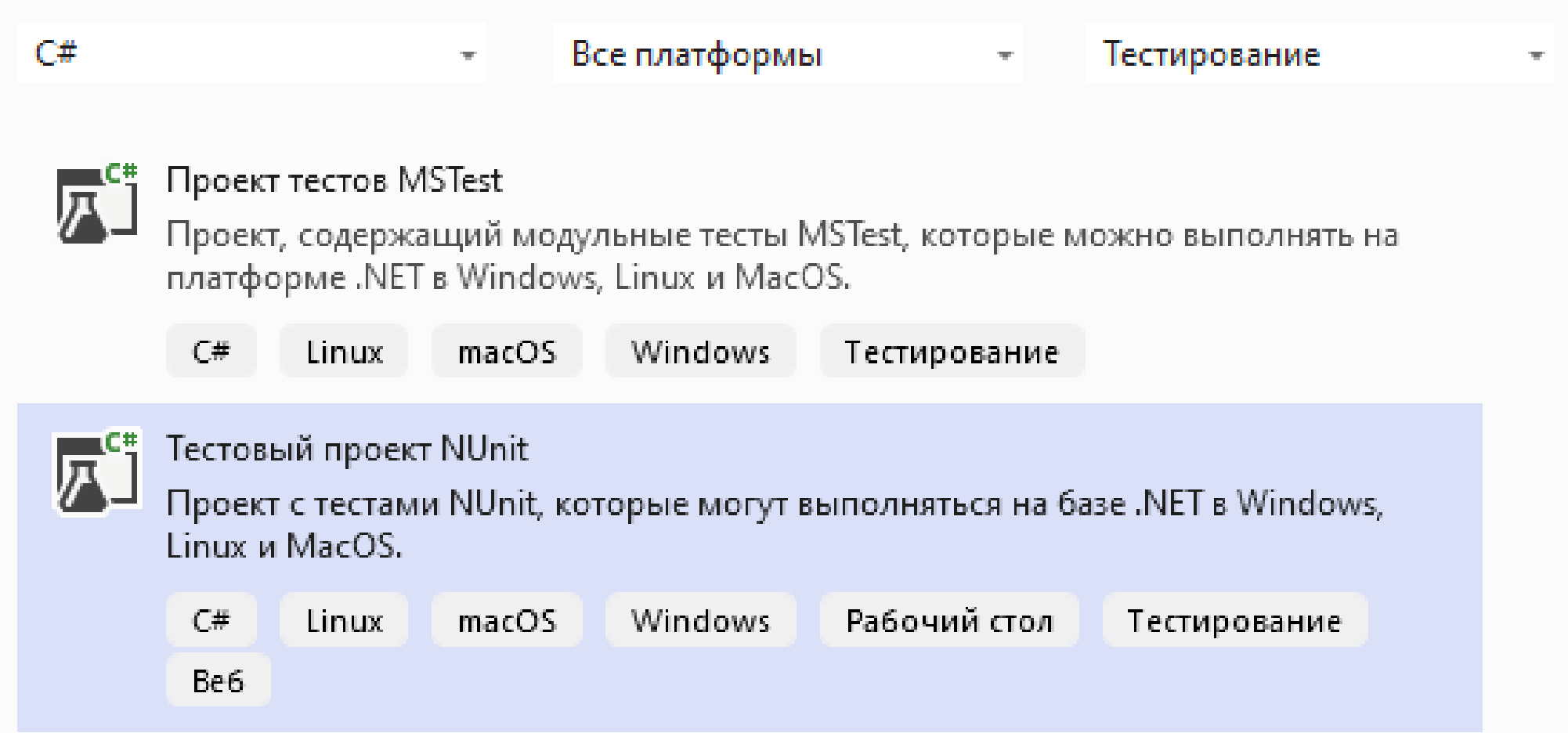
```
public class IntegerEquationSolver
    {
        public List<int> GetRootsOnInterval(int from, int to,
        Func<double, double> function)
        {
            var roots = new List<int>();

            for (int x = from; x <= to; x++)
            {
                if (Math.Abs(function(x)) <= 1e-12)
                    roots.Add(x);
            }
            return roots;
        }
    }
```

Добавим к решению тесты. Необходимо нажать ПКМ на Решение –>

Добавить –> Создать проект –>:

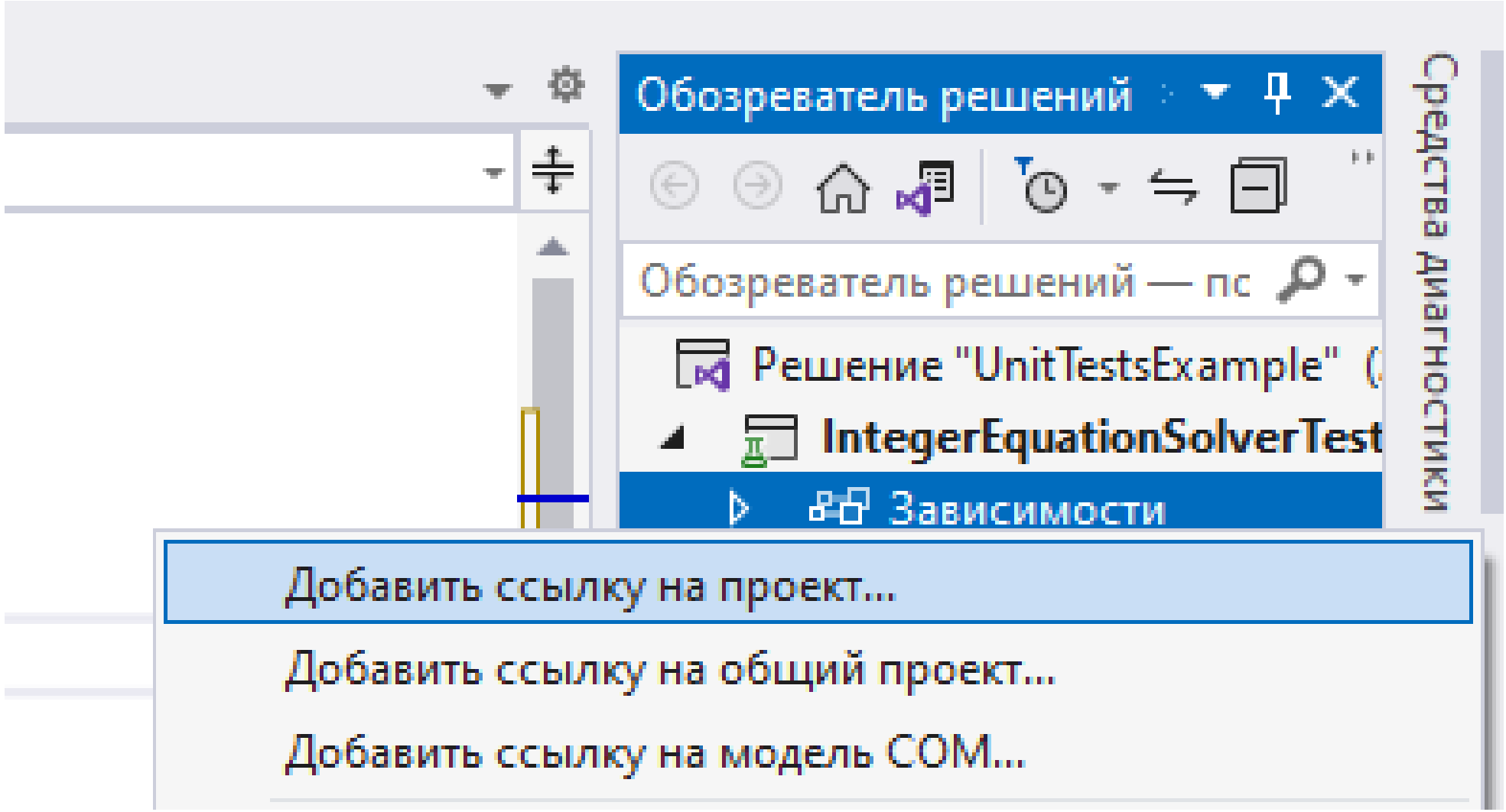


Выбрать C#, Все платформы, Тестирование. Из отобразившегося списка проектов выбрать «Тестовый проект NUnit»:

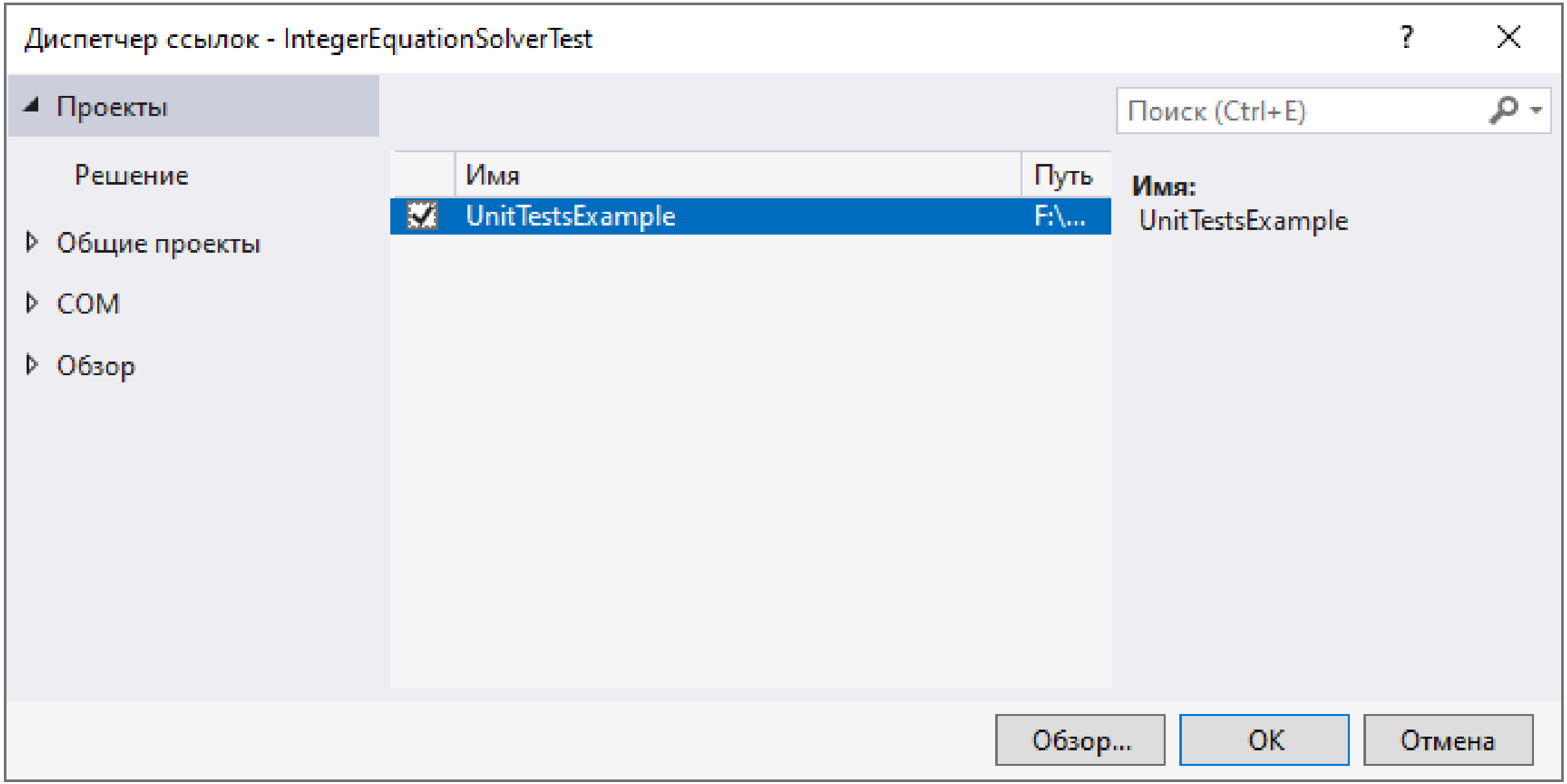


Подтверждаем и выбираем имя проекту, обычно тесты называются из названия класса, который будет тестироваться с припиской Test.

Создадим тесты для нашего класса поиска корней. Во-первых необходимо подключить наш проект к проекту тестов. Нажимаем ПКМ на Зависимости -> Добавить ссылку на проект.

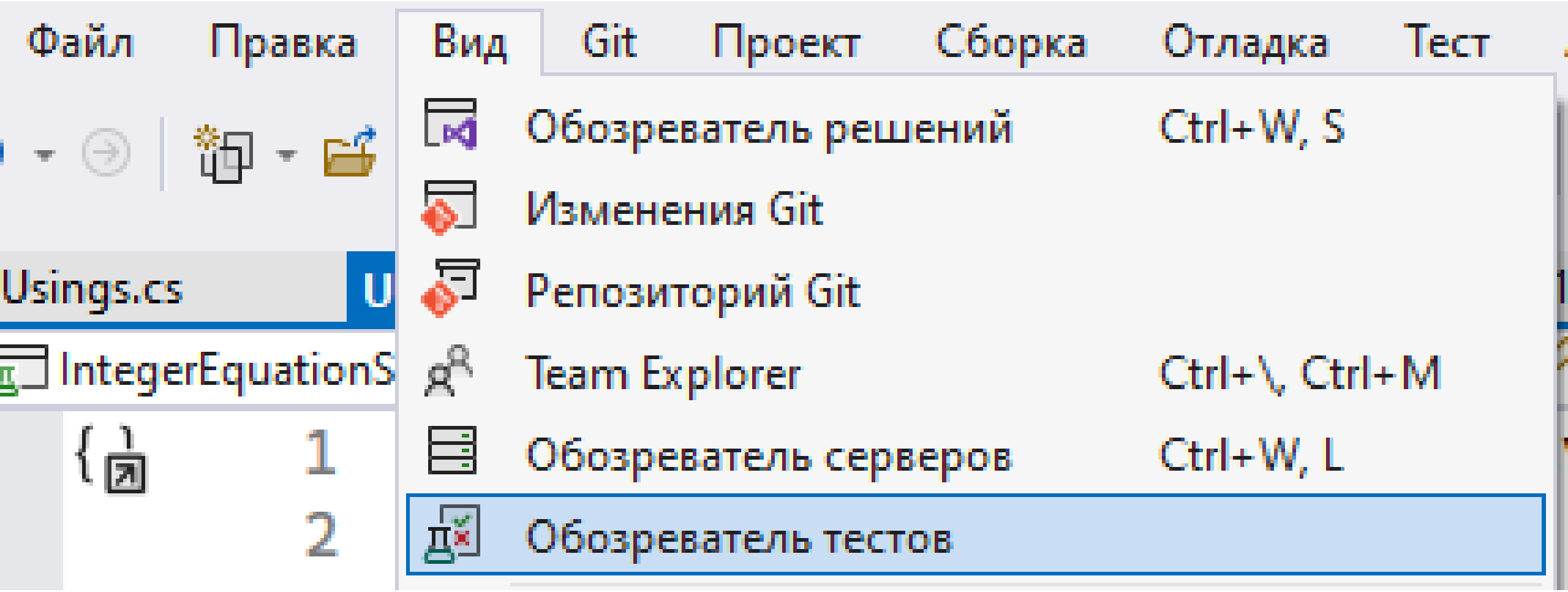


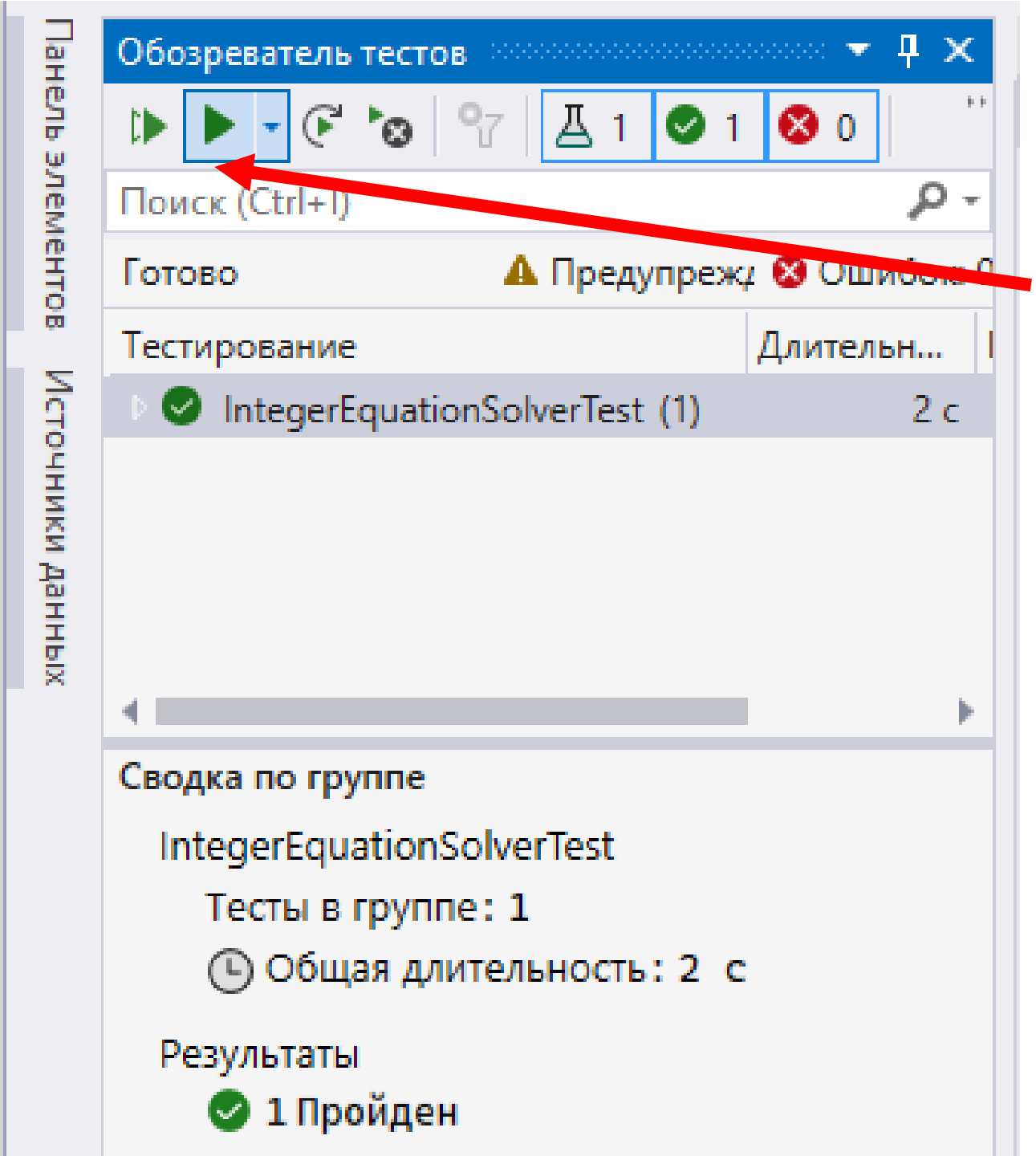
Выбираем нужный проект из списка.



```
using NUnit.Framework;
[TestFixture] // Атрибут класса с тестами
public class IntegerEquationSolverTests
{
    private IntegerEquationSolver _solver;

    [SetUp] // Выполняется ПЕРЕД каждым тестом
    public void Setup()
    {
        _solver = new IntegerEquationSolver();
    }
    [Test] // Атрибут самого теста
    public void GetRootsOnInterval_OneZeroRoot_ReturnsOnlyZero()
    {
        // Arrange
        Func<double, double> func = x => x * x;
        // Act
        var roots = _solver.GetRootsOnInterval(-10, 10, func);
        // Assert (Современный подход NUnit)
        Assert.That(roots, Has.Count.EqualTo(1));
        Assert.That(roots.First(), Is.EqualTo(0));
    }
}
```

Далее выберем: Вид -> обозреватель тестов.





В данном окне отображается основная информация по тестам, здесь находятся кнопки запуска и отладки тестов.

Синтаксис Assert.That

Современный NUnit использует читаемый Fluent-интерфейс (Constraint Model).

Базовый синтаксис: Assert.That(actual, Is.EqualTo(expected));

Примеры проверок:

Assert.That(result, Is.EqualTo(5)); — проверка на равенство

Assert.That(result, Is.Not.EqualTo(0)); — проверка на неравенство

Assert.That(result, Is.GreaterThan(10)); — больше чем

Assert.That(text, Does.Contain("error")); — проверка подстроки

Assert.That(obj, Is.Null); — проверка на null

Параметризованные тесты

Позволяют запустить один и тот же тест с разными наборами данных (техника эквивалентного разделения в действии).

```
[TestCase(-2, 2, 0)]   // x*x - 9 = 0 (нет корней от -2 до 2)
[TestCase(0, 4, 1)]    // корень 3
[TestCase(-4, 4, 2)]   // корни -3 и 3
public void GetRoots_WithDifferentIntervals_ReturnsCorrectRootsCount
    (int from, int to, int expectedCount)
{
    // Arrange & Act
    var roots = _solver.GetRootsOnInterval(from, to, x => x * x - 9);

    // Assert
    Assert.That(roots, Has.Count.EqualTo(expectedCount));
}
```

Тестирование коллекций

Позволяют запустить один и тот же тест с разными наборами данных (техника эквивалентного разделения в действии).

```
[TestCase(-4, 0, new[] { -3 })]
[TestCase(-4, 4, new[] { -3, 3 })]
public void GetRoots_ValidInterval_ReturnsExpectedValues
    (int from, int to, int[] expectedRoots)
{
    var roots = _solver.GetRootsOnInterval(from, to, x => x * x - 9);

    // Элементы совпадают (порядок не важен)
    Assert.That(roots, Is.EquivalentTo(expectedRoots));
}
```

Другие полезные проверки коллекций:

Assert.That(list, Is.Empty);

Assert.That(list, Is.Unique); — все элементы уникальны

Assert.That(list, Is.Ordered); — отсортирован ли список

Assert.That(list, Has.Member(5)); — содержит ли конкретный элемент

Тестирование исключений

Иногда ожидаемое поведение программы — это выдать ошибку (например, при делении на ноль или неверных аргументах).

```
[Test]
public void GetRoots_FromGreaterThanTo_ThrowsArgumentException()
{
    // Arrange
    int from = 10;
    int to = 0; // Ошибка: начало больше конца

    // Act & Assert
    Assert.Throws<ArgumentException>(() =>
    {
        _solver.GetRootsOnInterval(from, to, x => x * x);
    });
}
```

TDD (Test-Driven Development)

Разработка через тестирование (TDD)

Подход, при котором тесты пишутся *до* написания самого кода.

Цикл TDD (Red - Green - Refactor):

Red (Красный): Пишем тест для новой функции. Запускаем — он падает (т.к. кода еще нет).

Green (Зеленый): Пишем *минимальный* код, чтобы тест прошел успешно.

Refactor (Рефакторинг): Улучшаем написанный код, не боясь его сломать (ведь у нас есть работающий тест).